

UCS503 - Software Engineering

EduBridge

A Progressive Web Application for Small-Scale Coaching Institute Management

Submitted by:

1024170013 Abhijay Nagal
1024170120 Vansh Wadhwa
1024170115 Samarth Gulati
1024170010 Brahmansh Singh Sahni

Group: 3Q14 BE Third Year, CSE

Submitted to:

Sandeep Kaur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

1. Introduction

1.1 Background and Context

Small tuition centres and standalone coaching institutes make up a large part of supplementary education in India, but almost none of the learning management systems on the market are built for them. Products such as Google Classroom or Moodle assume a school-sized IT setup and administrative staff to run it. A tuition centre with one or two teachers and a few dozen students does not have either. In practice, these institutes run on WhatsApp groups for announcements, phone calls for attendance and fee reminders, and paper registers for records. This works, but it does not scale past a certain number of students and it leaves parents with almost no visibility into how their child is actually doing.

1.1.1 Problem Statement

The specific problem this project addresses is the absence of a lightweight, role-appropriate platform for three distinct groups of users at a small coaching institute: students, who need access to study material, quizzes and their own attendance and academic record; parents, who need to see their child's attendance, progress and fee status without having to call the teacher; and administration, who need to run the entire operation, upload material, mark attendance, record marks, track fees and broadcast notifications, from a single screen. Existing communication over WhatsApp is unstructured, not searchable, and puts the entire burden of manually sending reminders on the teacher. EduBridge replaces this with a single application that keeps this data structured and makes it available to the right person automatically.

1.2 Project Objectives

1. **Objective 1:** Design and build a single web-based platform with three role-based logins, student, parent and administration, that lets a small coaching institute manage attendance, study material, quizzes, marks and fee records without relying on WhatsApp groups or phone calls, delivered as a working prototype within the semester.
2. **Objective 2:** Implement the core administrative workflows, batch and student management, attendance marking, quiz creation with automatic grading, marks entry and fee tracking, so that any data the administration enters is immediately reflected on the student and parent dashboards without a manual sync step.
3. **Objective 3:** Package and deploy the application as an installable progressive web app hosted on Vercel with continuous deployment from GitHub, so that every push to the main branch is automatically built and released without a manual deployment step.

2. Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

EduBridge is a single-page application with one shared authentication system rather than three separate logins. Every account sits in one users table with a role field (student, parent or admin), and the application routes each role to its own dashboard and navigation after login. This keeps the authentication logic in one place while still giving each role a completely separate experience once inside the app.

All screens read and write through a single data-access module rather than talking to storage directly. This module currently persists data in the browser using IndexedDB, seeded with a realistic demo dataset on first launch, which means the whole prototype runs with no backend server and can be evaluated by opening the deployed link with no setup. Because every screen goes through this one module, moving to a shared backend at a later stage means rewriting that module only, not the components built on top of it.

Figure 2.1 shows the use case diagram for the system, covering the three actors and the behaviour available to each of them, including the include and extend relationships between login, quiz evaluation and the notification and fee reminder flows. Figure 2.2 shows the activity diagram for the login and role-dispatch flow, including the credential validation loop, the role-based split into the three dashboards, and the logout decision.

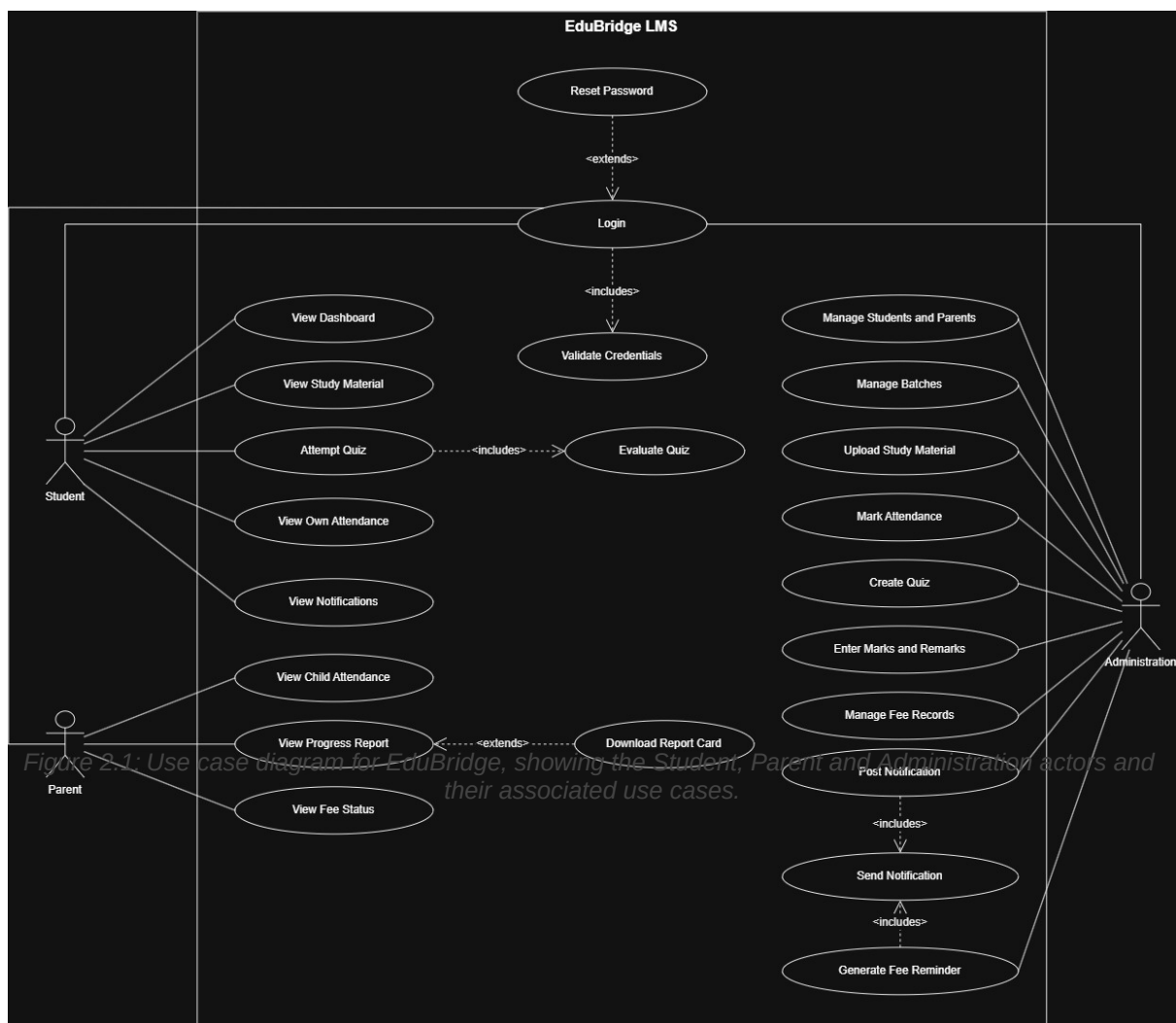


Figure 2.1: Use case diagram for EduBridge, showing the Student, Parent and Administration actors and their associated use cases.

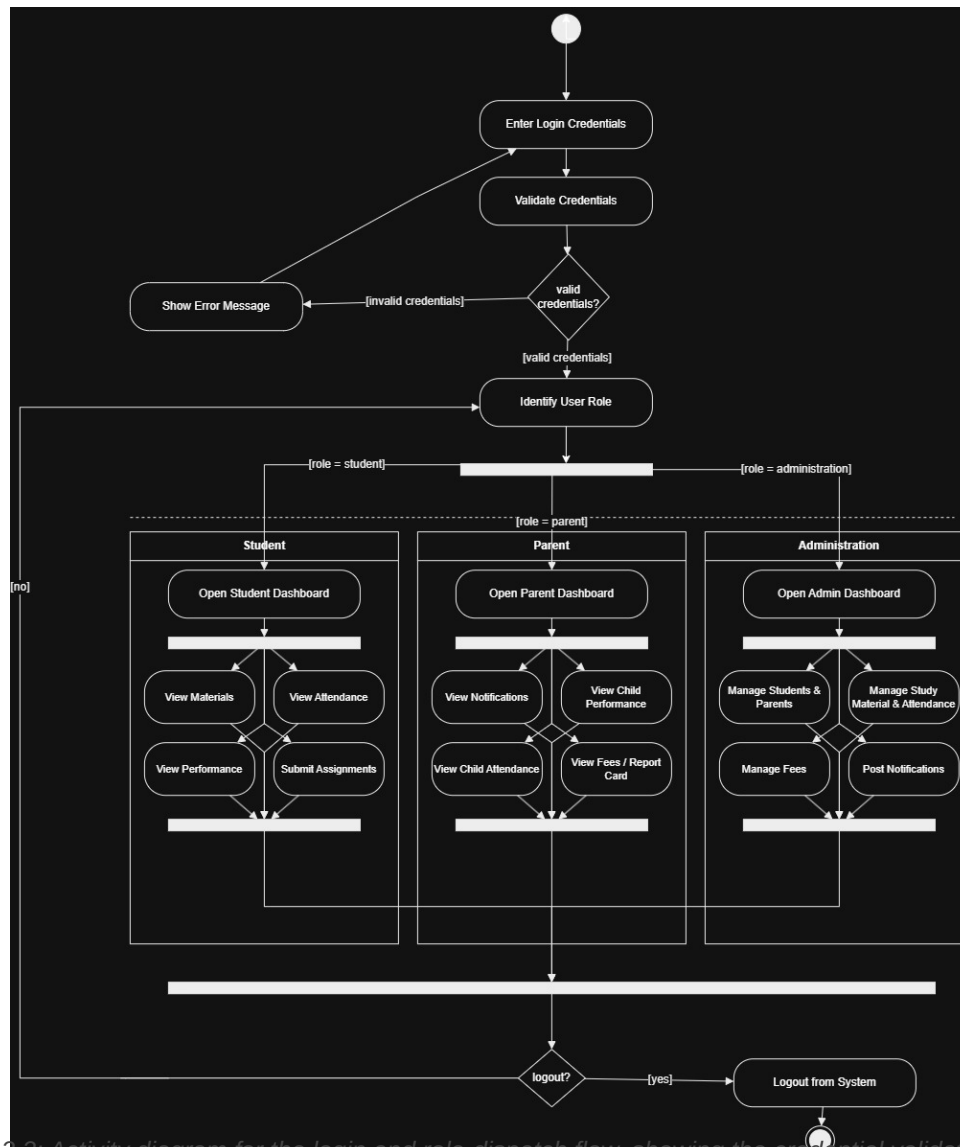


Figure 2.2: Activity diagram for the login and role-dispatch flow, showing the credential validation, role-based dashboard split and logout decision.

2.1.2 Technical Stack

- **Framework/Language:** React, built with Vite, written in plain JavaScript. Routing is handled by React Router.
- **Database:** IndexedDB, accessed through the idb-keyval helper library, behind a single data-access module.
- **Tools:** Git and GitHub for version control, Vercel for hosting with continuous deployment, sharp for generating PWA icons at build time.
- **Libraries:** Tailwind CSS for styling, Recharts for the attendance and progress charts, vite-plugin-pwa for the service worker and installability, following the Progressive Web App standard.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality

The authentication module checks a phone number and password against the users table and stores the resulting session in local storage, with route guards that redirect a signed-in user to their own role's dashboard and log out any user whose underlying student or parent record has been removed.

The attendance module lets the administration mark an entire batch in one screen, with a three-state toggle per student (present, late, absent) and an "all present" shortcut, since a teacher marking thirty students by hand needs this to be fast. Saving overwrites any existing record for that batch and date rather than creating duplicates, which was verified directly with an automated test.

The quiz module supports multiple-choice questions with a single correct option per question. A student can attempt a quiz once; on submission the module computes the score against the stored correct answers and stores the attempt, after which the student sees a full answer review showing which options were correct and which they chose.

2.2.2 Module 2: Secondary Features

The notification module lets administration post an announcement targeted at everyone, students only or parents only, and optionally scoped to a single batch. Each user's notification list is filtered by their role and batch membership, so a parent never sees a student-only notice and vice versa. An unread count is tracked per user and shown as a badge on the navigation bar.

The fee module tracks a due amount and date per student, automatically flags an entry as overdue once its due date passes, and lets administration record a payment against it. A single button posts one reminder notification to all parents with an overdue fee, which is the manual version of the scheduled reminder job described in the original project plan.

The report card module stores marks and a remark per subject per test, and renders the same data as a trend line across tests and a bar chart per subject on both the student's own progress screen and the parent's view of their child, using Recharts.

Because the whole application is registered as a service worker with a web app manifest, it can be installed to a phone's home screen from the browser and opens without any browser chrome, which was a specific requirement for this project.

3. Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** The data-access module was exercised outside the browser with a Node script that stubs IndexedDB in memory. It covers authentication with correct and incorrect credentials, role-based notification filtering, quiz scoring on both a fully correct and a fully incorrect attempt, attendance overwrite behaviour, fee overdue flagging, payment recording, and cascading deletes when a student or batch is removed.
- **Integration Testing:** The production build was served locally and every top-level route, including deep links such as `/student/quizzes`, was checked to confirm it returns a valid page rather than a routing error, which specifically verifies that the single-page application rewrite rule on Vercel is configured correctly.
- **Performance Testing:** The production bundle was inspected after enabling manual code-splitting, separating the charting library into its own chunk so it is not loaded on screens that do not need it.
- **User Testing:** Not carried out yet at this stage of the project. This is listed under future work.

3.2 Performance Metrics

Table 3.1 reports figures taken directly from the production build and from the automated checks described above, rather than simulated targets.

Metric	Result
Automated data-layer checks passed	38 / 38
Core routes returning a valid page	7 / 7
Main application bundle (gzipped)	25.9 kB
Charting library bundle (gzipped, lazy-loaded)	105.6 kB
React and routing bundle (gzipped)	53.5 kB
PWA icon sizes generated	192px, 512px, maskable 512px

Table 3.1: Build and test results for the current prototype

3.3 Analysis

The first two objectives, a working three-role platform and the core administrative workflows, have been met in full for this evaluation. The third objective, packaging

and deployment, was also met and slightly exceeded: the application is live on Vercel with continuous deployment already wired up, and it is installable as a home screen app on both Android and iOS, which was treated as a stretch goal at the planning stage rather than a requirement.

The main challenge encountered was choosing a data layer for this stage of the project. The original plan called for a shared Supabase backend, but setting that up correctly, including authentication rules and row-level security, was judged too large a task to complete safely in the time available before this evaluation without risking a broken submission. The decision taken was to build a fully functional local data layer instead, isolated behind one module, so that every feature works end to end for grading and demonstration purposes, with the understanding that moving to a shared backend is a contained follow-up task rather than a rewrite. The direct consequence of this decision is that data is currently local to each browser rather than shared across devices, which is documented as a known limitation rather than left as a silent gap.

4. Conclusion

4.1 Summary of Work

This evaluation covers the full first version of EduBridge: a three-role progressive web application covering study material, quizzes, attendance, report cards, fee tracking and targeted notifications, deployed with continuous integration from GitHub to Vercel. All three stated objectives were met within the current stage of the project.

4.2 Key Findings

- **Finding 1:** Modelling all three roles around one authentication table with a role field, rather than three separate login systems, made the role-based routing and access rules considerably simpler to implement and to reason about than three independent systems would have been.
- **Finding 2:** Routing every screen through a single data-access module, rather than letting individual components read storage directly, made it possible to write meaningful automated tests for the application logic even without a browser test environment available in this setup.
- **Finding 3:** A local-first data layer was sufficient to deliver a fully working, installable prototype without any backend infrastructure, which was not the original plan but turned out to be a reasonable way to de-risk the delivery timeline for this evaluation.

4.3 Future Work and Recommendations

1. Replace the local IndexedDB data layer with a shared Supabase backend for authentication, the database and file storage, so that data is consistent across every device rather than local to one browser.
2. Add real file uploads for study material, since the current version only supports written notes and external links.
3. Integrate an SMS or WhatsApp Business API channel for fee reminders and class-cancellation notices, since in-app notifications alone are unlikely to reach parents who do not open the app daily.
4. Extend automated testing to cover the user interface itself once a browser testing setup is available, rather than only the underlying data logic.
5. Carry out a short round of user testing with an actual tuition teacher and a small batch of students to validate the attendance marking and quiz flows under real conditions before any real deployment.

4.4 Final Remarks

Building EduBridge around a single, swappable data-access layer turned out to be the most useful early decision in the project. It meant the team could commit to a local-first version for this stage without locking the architecture into that choice permanently, and it kept every screen simple to build because none of them needed to know how or where the data was actually stored.

Bibliography

Meta Open Source. React – The Library for Web and Native User Interfaces. 2024. <https://react.dev>

VoidZero Inc. Vite – Next Generation Frontend Tooling. 2024. <https://vitejs.dev>

Tailwind Labs. Tailwind CSS – A Utility-First CSS Framework. 2024. <https://tailwindcss.com>

MDN Contributors. Progressive Web Apps. MDN Web Docs, 2024. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps

Archibald, Jake. idb-keyval – A Super-Simplified Promise-Based Keyval Store. 2023. <https://github.com/jakearchibald/idb-keyval>

